HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

BÁO CÁO TUẦN 8

Đề tài: Bài toán nhận diện biển số xe

Giảng viên hướng dẫn:	Kim Ngọc Bách
Nhóm lớp	11
Họ và tên sinh viên	Đinh Hoàng Vũ
Mã sinh viên	B23DCCN941

Hà Nội – 2026

MỤC LỤC

1. Chuyển đổi mô hình YOLO 3
1.1. Quy trình chuyển đổi..... 3
1.2. Kết quả 4
2. Chuyển đổi PaddleOcr 4
2.1. Các bước thực hiện 4

BÁO CÁO TUẦN HỌC PHẦN THỰC TẬP CƠ SỞ LẦN 7

MỤC TIÊU: Chuyển đổi mô hình YOLO và mô hình nhận dạng ký tự của PaddleOCR từ định dạng Paddle Inference sang ONNX để cải thiện hiệu năng nhận dạng trong hệ thống real-time.

1. Chuyển đổi mô hình YOLO

Mô hình YOLO sau khi huấn luyện thường được lưu dưới dạng .pt đây là định dạng đặc trưng của PyTorch. Tuy nhiên, khi triển khai trong môi trường thực tế, đặc biệt trên CPU hoặc các nền tảng không sử dụng trực tiếp Pytorch mà sử dụng định dạng ONNX để tăng tốc độ suy luận.

1.1. Quy trình chuyển đổi

1. Sử dụng thư viện Ultralytics để xuất mô hình từ .pt sang .onnx

```
from ultralytics import YOLO
model = YOLO("best_pl_detecton.pt")
model.export(format="onnx", dynamic=True, opset=12)
```

Sau khi chuyển đổi ta thu được tệp mô hình: best_pl_detection.onnx, mô hình này giữ nguyên khả năng phát hiện biển số so với mô hình gốc, đồng thời có thể sử dụng trực tiếp với ONNX Runtime để suy luận hiệu quả hơn.

2. Cài đặt thư viện onnxruntime

```
pip install onnxruntime
```

3. Xây dựng hàm letterbox

Khi chuyển đổi mô hình YOLO sang định dạng ONNX, có một vấn đề là mô hình yêu cầu một kích thước đầu vào cố định (ví dụ 640 x 640) trong khi hình ảnh thực tế lại có kích thước và tỉ lệ khung hình rất đa dạng. Nếu sử dụng hàm cv2.resize thông thường để ép một hình ảnh chữ nhật về hình vuông, vật thể sẽ bị so giãn. Hậu quả là kết quả suy luận của mô hình sang hình ảnh thực tế sẽ không còn chính xác. Với mô hình YOLO nó đã tích hợp sẵn pipeline tiền xử lý để khắc phục vấn đề này. Tuy nhiên, với ONNX nó chỉ là "bộ máy tính toán" thuần túy, nó chỉ nhận vào các con số (tensor) và trả ra các con số và không có khả năng tiền xử lý ảnh. Do đó, cần phải xây dựng hàm letterbox để chuẩn hóa dữ liệu đầu vào và khôi phục tọa độ về ảnh gốc

```
def letterbox(img, new_shape=(640, 640)):
    shape = img.shape[:2]
    r = min(new_shape[0] / shape[0], new_shape[1] / shape[1])
    new_unpad = (int(round(shape[1] * r)), int(round(shape[0] * r)))
    dw, dh = (new_shape[1] - new_unpad[0]) / 2, (new_shape[0] - new_unpad[1]) / 2
    img = cv2.resize(img, new_unpad, interpolation=cv2.INTER_LINEAR)
    img = cv2.copyMakeBorder(img, int(round(dh - 0.1)), int(round(dh + 0.1)),
                             int(round(dw - 0.1)), int(round(dw + 0.1)),
                             cv2.BORDER_CONSTANT, value=(114, 114, 114))
    return img, r, (dw, dh)
```

Hàm letterbox thực hiện việc chuyển đổi hình ảnh về kích thước chuẩn của mô hình ONNX mà không làm biến dạng vật thể. Bằng cách kết hợp giữa việc thay đổi kích thước và giữ nguyên tỷ lệ và kỹ thuật thêm viền (padding) màu xám trung tính, hàm đảm bảo tính nhất quán của dữ liệu đầu vào và cung cấp các thông số cần thiết để khôi phục tạo độ vật thể ở bước hậu xử lý.

4. Xây dựng hàm hậu xử lý

Do kết quả trả về từ mô hình ONNX là các Tensor thô dưới dạng ma trận toán học, hàm detect_plates đã được xây dựng để thực hiện các nhiệm vụ hậu xử lý:

- **Giải mã tọa độ:** Chuyển đổi định dạng Center-XY sang định dạng Corner-XY phù hợp với OpenCV.
- **Hủy Padding và Scale:** Sử dụng các tham số ratio và pad từ hàm letterbox để ánh xạ chính xác vị trí biển số từ không gian của mô hình về không gian của ảnh gốc, đảm bảo khung nhận diện không bị lệch khi hiển thị.
- **Lọc nhiễu bằng NMS:** Áp dụng thuật toán Non-Maximum Suppression để loại bỏ các vùng nhận diện chồng lấn, chỉ giữ lại kết quả tối ưu nhất cho từng biển số xe.

Kết quả, hàm trả về danh sách tọa độ $[left, top, width, height]$ chính xác trên ảnh thực tế, sẵn sàng cho việc cắt vùng ảnh biển số để đưa vào mô hình nhận dạng ký tự (OCR) ở bước tiếp theo.

1.2. Kết quả

Mô hình sau khi chuyển sang định dạng ONNX có khả năng phát hiện đối tượng tương đương mô hình gốc, đảm bỏa độ chính xác không bị suy giảm sau quá trình chuyển đổi. Tốc độ suy luận được cải thiện rõ rệt, đặc biệt khi triển khai trên CPU, thời gian xử lý cho mỗi ảnh cũng giảm đáng kể khoảng 2-4 lần.

2. Chuyển đổi PaddleOcr

2.1. Các bước thực hiện

1. Cài đặt các thư viện liên quan

```
pip install paddleocr paddlepaddle
pip install paddle2onnx
```

2. Thực hiện chuyển đổi sang ONNX

```
paddle2onnx --model_dir . --model_filename inference.pdmodel --params_filename
inference.pdiparams --save_file ppocr_rec.onnx --opset_version 14
```

3. Tối ưu model

```
python -m onnxsim ppocr_rec.onnx ppocr_rec_sim.onnx
```

4. Kết quả

File mô hình mới được tạo “ppocr_rec_sim.onnx” dùng để đọc ký tự, thực hiện tương tự ta có thêm mô hình “ppocr_det_sim.onnx” dùng để phát hiện vùng kí tự